Binary Search Trees: Search, Insert, Delete

Programming and Data Structures — Week 7, Lecture 2

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to state the binary search tree property precisely and check whether a given tree satisfies it; implement BST search, insertion, and deletion entirely from scratch; trace all three operations completely, including the three distinct cases that arise during deletion; and explain why every BST operation costs $O(h)$, where h is the tree's height, connecting directly to Lecture 1's height/size relationship.

Outline

Today covers a phone-book-with-branches analogy motivating the BST property; a precise statement of that property; search, traced completely; insertion, traced completely; deletion, which requires handling three distinct structural cases, each traced in turn; why every BST operation costs $O(h)$, and the height-guarantee problem this leaves unresolved; the MTech-track formal treatment of the in-order successor, needed for one of the deletion cases; an in-class quiz; and a summary.

The phone-book-with-branches analogy

Week 1 Lecture 4 introduced binary search on a sorted array: repeatedly halving the range of values still under consideration. A binary search tree makes this halving *structural*, built directly into the shape of the data: at every node, everything in the left subtree is guaranteed smaller, and everything in the right subtree is guaranteed larger. Unlike Week 2's sorted array, where inserting a new value required an O(n) shift to maintain sorted order, a BST simply attaches a new node at the position the property dictates — no shifting of existing nodes required at all.

The BST property, defined precisely

The binary search tree property states that, for every single node in the tree, all values in its left subtree are strictly less than that node's own value, and all values in its right subtree are strictly greater. This must hold recursively at every node in the tree, not merely at the root — a common mistake when first checking whether a tree is a valid BST is to verify only that a node's immediate children are correctly ordered, missing that a deeper descendant could still violate the property relative to an ancestor further up the tree.

Checking the property: a common mistake, exposed

```
      5
     / \
    3   8
       / \
      4   9      <-- INVALID: 4 is in 5's right subtree, but 4 < 5!
```

This tree looks locally correct at every parent-child pair: 4 is less than its immediate parent 8, satisfying that local check. But the BST property is a statement about entire *subtrees*, not just immediate parent-child pairs — 4 sits within the right subtree of the root, 5, and the property requires every value in that subtree to exceed 5, which 4 violates. Correctly validating a BST requires passing down a valid range (a minimum and maximum bound) at each recursive step, not merely comparing a node against its immediate parent.

Search, in code

```
def bst_search(node, target):
    if node is None or node.value == target:
        return node
    if target < node.value:
        return bst_search(node.left, target)
    else:
        return bst_search(node.right, target)
```

BST search mirrors binary search from Week 1 Lecture 4 almost exactly: at every node, one comparison against the target determines which single subtree could possibly contain it, and the other subtree is never visited at all — not because it has been ruled out by scanning, but because the BST property structurally guarantees it cannot contain the target. This is the entire reason a BST is useful: the property eliminates the need to ever examine roughly half the remaining data at each step, exactly mirroring array-based binary search's halving.

Tracing search for 6 in a concrete tree

```
Tree:        8
           /   \
          3     10
         / \      \
        1   6      14

search(root, 6):
  6 vs 8: 6<8 → go left, to node 3
  6 vs 3: 6>3 → go right, to node 6
  6 vs 6: MATCH → return node 6
```

Tracing the search for 6 on Lecture 1's example tree: starting at the root, 8, since $6 < 8$, the search moves left to node 3, never examining the entire right subtree rooted at 10 at all — that subtree, containing 14, is structurally guaranteed not to contain 6. At node 3, since $6 > 3$, the search moves right to node 6, where it finds an immediate match. Only 3 comparisons were needed to search a 6-node tree — this lecture's later analysis makes precise why this scales as $O(h)$, the tree's height, not $O(n)$.

Insert, in code

```python
def bst_insert(node, value):
    if node is None:
        return TreeNode(value)          # base case: create the new node here
    if value < node.value:
        node.left = bst_insert(node.left, value)
    elif value > node.value:
        node.right = bst_insert(node.right, value)
    return node    # unchanged if value already exists
```

Insertion follows precisely the same path search would follow, comparing the new value against each node and moving left or right accordingly, until reaching a `None` — an empty position that respects the BST property for this value. A new node is created exactly there and returned up through the recursion, reattaching it to its parent. Unlike Week 2's sorted array, which required shifting every subsequent element to insert in sorted order, this requires creating only one new node and updating one parent reference — no shifting of any existing node at all.

Tracing insert(7) on the same tree

```
insert(root=8, 7):
  7<8 → recurse left into node 3
  insert(3, 7): 7>3 → recurse right into node 6
    insert(6, 7): 7>6 → recurse right into node.right (None)
      insert(None, 7): create TreeNode(7), return it
    node 6's right = new node 7
  node 3's right unchanged (still points to 6)
Result:        8
              /   \
             3     10
            / \      \
           1   6      14
               \
                7
```

Tracing `insert(7)`: following the same comparisons search would make, the recursion travels 8 → 3 → 6, and at node 6 finds that 7 belongs to the right, which is currently `None`. A new node holding 7 is created there and attached as node 6's right child. Every node along the path (8, 3, 6) is revisited during the recursive unwinding, but none of them actually change — only node 6 gains a new right child, exactly one structural change for one inserted value.

Delete: the easy case, a leaf

```python
def bst_delete(node, value):
    if node is None:
        return None
    if value < node.value:
        node.left = bst_delete(node.left, value)
    elif value > node.value:
        node.right = bst_delete(node.right, value)
    else:
        # FOUND the node to delete
        if node.left is None and node.right is None:
            return None                  # Case 1: leaf - just remove it
        # (cases 2 and 3 follow)
    return node
```

Deletion is the most involved of the three operations, requiring three distinct cases. Case 1, the simplest: the node to delete is a leaf, with no children at all. Detaching it is trivial — returning `None` up to the

parent, which overwrites whichever reference (`left` or `right`) pointed to the deleted node. No other part of the tree is affected, and the BST property is trivially preserved, since removing a leaf cannot disturb any ordering relationship among the remaining nodes.

Delete: one child

```
        if node.left is None:
            return node.right     # Case 2a: only a right child - it takes the deleted node's
        if node.right is None:
            return node.left      # Case 2b: only a left child - it takes the deleted node's
```

Case 2 handles a node with exactly one child: that single child, along with its entire subtree, simply moves up to occupy the deleted node's former position, returned directly to the parent in place of the deleted node. The BST property is automatically preserved, because the surviving child's entire subtree was already correctly ordered relative to everything above the deleted node — deleting a node with one child does not require re-examining any other part of the tree at all.

Delete: two children — the genuinely hard case

```
        # Case 3: two children - replace with the in-order SUCCESSOR
        successor = node.right
        while successor.left is not None:
            successor = successor.left      # smallest value in the right subtree
        node.value = successor.value         # copy successor's value up
        node.right = bst_delete(node.right, successor.value)   # then delete the successor
```

Case 3, a node with two children, is genuinely more involved: neither child can simply take the deleted node's place without breaking the tree's connectivity, since both entire subtrees need a home. The standard fix finds the node's in-order successor — the smallest value in its right subtree, found by following `left` references as far as possible — and copies that successor's value into the position being deleted. The successor node itself, now duplicated, is then deleted from its original position; because the in-order successor (the smallest value in a subtree) can never itself have a left child, this recursive deletion always reduces to Case 1 or Case 2, never back to Case 3.

Tracing delete(3) — a two-children case, completely

```
Tree:         8
            /   \
           3     10
          / \      \
         1   6      14

delete(3): node 3 has TWO children (1 and 6)
  successor = node 3's right subtree (rooted at 6), walk .left: 6 has no left → successor = 6
  Copy: node (formerly 3)'s value becomes 6
  Delete original node 6 from the right subtree: 6 is a leaf there → Case 1, removed

Result:         8
              /   \
             6     10
            /         \
           1           14
```

Tracing `delete(3)` on Lecture 1's example tree: node 3 has two children, 1 and 6, triggering Case 3. The in-order successor is found by entering the right subtree (rooted at 6) and following `left` references as far as possible — since 6 has no left child, 6 is itself the successor. The value 6 is copied into the node being deleted (which now holds the value 6, still positioned where 3 used to be), and the original node holding 6 is then deleted from its old position — where it is a leaf, reducing cleanly to Case 1. The final tree correctly preserves the BST property throughout.

Why every operation is O(h)

All three operations covered in this lecture share a single structural property: each one follows exactly one path from the root downward, never needing to explore both subtrees of any node simultaneously. The cost of each operation is therefore bounded by the length of that path, which can be at most the tree's height, h — not its total size, n. This is precisely why Lecture 1's height/size relationship matters so much: if the tree happens to be balanced, with $h = \Theta(\log n)$, every operation costs $O(\log n)$, matching Week 1's array-based binary search exactly. But if the tree has degenerated toward a chain, with $h = \Theta(n)$, every operation costs $O(n)$ — offering no advantage whatsoever over Week 4's plain linked list, despite the additional complexity of maintaining a tree structure at all.

The unresolved problem this leaves

Nothing in the `bst_insert` code as written this lecture prevents a tree from degenerating toward the worst case. Inserting values in already- sorted order — 1, then 2, then 3, and so on — produces exactly

the degenerate chain illustrated in Lecture 1's balanced-versus-degenerate figure, since every new value is always greater than everything already present, always attaching as a right child with no branching ever occurring. This is precisely the motivation for self-balancing trees, which actively restructure themselves during insertion and deletion to guarantee $h = O(\log n)$ regardless of insertion order — the subject of the next lecture.

MTech addition — the in-order successor, formally

The in-order successor of a node is formally defined as the value that would immediately follow it in an inorder traversal, per Lecture 1. When a node has a right subtree, its successor is always the leftmost node within that subtree — exactly the search this lecture's deletion code performs, since the leftmost node in any subtree holds that subtree's smallest value. When a node has no right subtree at all, finding its successor requires walking upward toward the nearest ancestor for which the original node lies within that ancestor's left subtree — this case requires either an explicit `parent` reference on each node, or tracking the path of ancestors visited during the original search, in a stack, directly reusing Week 3's structure.

Exercise

As an exercise, insert the sequence `[5, 3, 8, 1, 4, 7, 9]` into an initially empty BST one value at a time, drawing the tree's structure after each individual insertion. Then delete the root, 5, which has two children, and draw the resulting tree, explicitly identifying the in-order successor used and confirming which of this lecture's three deletion cases the successor's own removal reduces to.

Quiz — is this a valid BST?

Given this tree — root 10, left child 5 and right child 15, and 5's own children being 1 and 12 — determine before checking the answer whether this is a valid binary search tree, and explain your reasoning precisely.

Quiz — answer

This is not a valid binary search tree. The node holding 12 sits within the left subtree of the root, 10, but $12 > 10$, violating the property that everything in the root's left subtree must be less than the root. Locally, 12 is correctly placed relative to its immediate parent, 5, since $12 > 5$ satisfies that one relationship — but as this lecture warned earlier, the BST property must hold relative to *every* ancestor a node has, not merely its immediate parent, and this tree fails that broader check.

Summary

The binary search tree property requires every node's left subtree to hold entirely smaller values and its right subtree entirely larger values, recursively, at every single node, not merely immediate parent-child pairs. Search and insertion both follow a single root-to- target path; deletion requires handling three distinct structural cases, the hardest being a node with two children, resolved by substituting its in-order successor. Every operation costs $O(h)$, the tree's height — excellent when the tree is balanced, but no better than Week 4's linked list when it has degenerated. Critically, nothing in the plain BST insertion algorithm covered this lecture prevents such degeneration — inserting already-sorted data produces the worst case directly and reliably. Next lecture introduces self-balancing trees, which actively restructure themselves during insertion and deletion specifically to prevent this degeneration from ever occurring.